MBLP131 · Hafta 2 · Ders Notları

İşletim Sistemleri Temelleri

Tanımlar, Hedefler, Bileşenler ve İşletim Sistemi Yapıları

İşletim sistemi tam olarak nedir, ne işe yarar ve nasıl inşa edilir?

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Ders Telegram kanalı: `[ders grubu bağlantısı]`

Bu haftanın sonunda yapabilmeniz gerekenler

1. İşletim sisteminin ve *çekirdeğin* (kernel) kesin bir tanımını vermek.
2. İşletim sistemine iki klasik bakışı açıklamak: *genişletilmiş makine* ve *kaynak yöneticisi*.
3. İşletim sisteminin üç tasarım hedefini saymak ve bu hedeflerin neden çatıştığını açıklamak.
4. Bir bilgisayar sisteminin dört bileşenini, kesmelerin rolünü, bellek hiyerarşisini ve ikili mod çalışmasını anlatmak.
5. Basit, monolitik, katmanlı, mikroçekirdek, modüler ve hibrit işletim sistemi yapılarını her birine bir örnek vererek karşılaştırmak.
6. Sistem çağrısı yapan küçük C programlarını okuyup çalıştırmak ve bir sistem çağrısının maliyetini ölçmek.

İçindekiler

1 İşletim Sistemi Tam Olarak Nedir? 2

1.1 İki klasik bakış . . . 3

1.2 Çekirdek, sistem programları ve uygulama programları . . . 3

2 İşletim Sisteminin Tasarım Hedefleri 4

2.1 Hedefler çatışır . . . 4

3 Bilgisayar Sisteminin Bileşenleri 5

3.1 Donanımın düzeni . . . 5

3.2 Kesmeler . . . 6

3.3 Bellek hiyerarşisi . . . 7

3.4 İkili mod çalışması . . . 7

4 Uygulama: Bir Programın Çekirdeği Çağırmasını İzlemek **8**

4.1 Örnek 1: kütüphane çağrısı ve sistem çağrısı 8

4.2 Örnek 2: bir sistem çağrısı ne kadar pahalı? 8

5 İşletim Sistemi Yapıları **10**

5.1 Basit yapı: MS-DOS 10

5.2 Monolitik çekirdek: UNIX ve Linux 11

5.3 Katmanlı yaklaşım 11

5.4 Mikroçekirdek 12

5.5 Modüler çekirdek: yüklenebilir çekirdek modülleri 12

5.6 Hibrit sistemler 14

5.7 Karşılaştırma 14

6 Özet **15**

7 Sözlük **15**

8 Kendiniz Deneyin **16**

9 Tekrar Soruları **16**

10 İleriye Bakış: 3. Hafta **17**

1 İşletim Sistemi Tam Olarak Nedir?

Geçen hafta gayriresmî bir tanım kullandık: *işletim sistemi, ilk başlayan ve geri kalan her şeyi yöneten programdır*. Ders kitapları, işletim sistemine her biri farklı bir açıdan bakan daha kesin tanımlar verir.

Bakış açısı	Tanım
Konum	Bilgisayarın kullanıcısı ile bilgisayar donanımı arasında *aracılık eden* (intermediary) bir program.
Süreklilik	Bilgisayarda her an çalışan tek program; genellikle **çekirdek** (kernel) olarak adlandırılır. Geri kalan her şey ya bir sistem programı ya da bir uygulama programıdır.
İşlev	Programların yürütülmesini denetleyen ve bilgisayarın kaynaklarını (işlemci, bellek, dosyalar ve aygıtlar) yöneten yazılım.

Bu dersin çalışma tanımı

İşletim sistemi, donanımı yöneten ve programlara üzerinde çalışabilecekleri daha basit, daha güvenli bir makine sunan yazılım katmanıdır. Bellekte sürekli bulunan özüne **çekirdek** (kernel) denir.

1.1 İki klasik bakış

Andrew Tanenbaum işletim sistemini birbirine zıt iki yönden anlatır.

Yukarıdan aşağıya: genişletilmiş makine olarak işletim sistemi. Gerçek donanım karmaşık ve kullanışsızdır. Bir disk, sektörlere bölünmüş manyetik plakalardan ya da flaş hücrelerinden oluşur; kendi komutları, hata kodları ve zamanlama kuralları vardır. Neredeyse hiçbir programcı bunlarla uğraşmak istemez. İşletim sistemi bu ayrıntıları temiz **soyutlamaların** (abstraction) arkasına gizler: sektörler yerine *dosyaları*, yazmaçlar ve kesme hatları yerine *süreçleri* (process), video belleği yerine *pencereleri* görürsünüz. Bu açıdan bakıldığında işletim sistemi, gerçek makineyi programlaması kolay, daha hoş, "genişletilmiş" bir makineye dönüştürür.

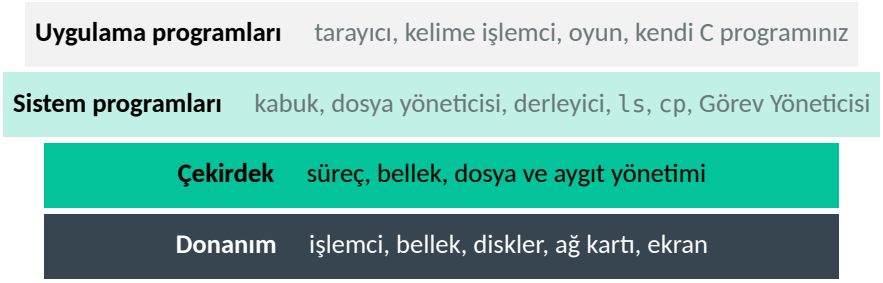
Aşağıdan yukarıya: kaynak yöneticisi olarak işletim sistemi. Birçok program ve kullanıcı aynı donanım için yarışır. Kimin neyi ne zaman alacağına işletim sistemi karar verir. Bunu kaynakları iki biçimde **çoğullayarak** (multiplexing), yani paylaştırarak yapar:

- **Zaman çoğullama (time multiplexing):** programlar sırayla kullanır. İşlemci bir programı birkaç milisaniye çalıştırır, sonra bir başkasını; yazıcı bir belgeyi basar, sonra bir sonrakini.
- **Alan çoğullama (space multiplexing):** kaynak parçalara bölünür. Ana bellek aynı anda birkaç programı tutar; bir disk, birçok kullanıcıya ait dosyaları barındırır.

İki bakış da aynı yazılımı anlatır. Bu dersin büyük kısmı (süreçler, işlemci zamanlaması, bellek, dosyalar) işletim sistemini bir kaynak yöneticisi olarak inceler.

1.2 Çekirdek, sistem programları ve uygulama programları

İşletim sistemiyle birlikte gelen her şey çekirdek değildir.



Şekil 1: Tipik bir bilgisayardaki yazılım katmanları.

- **Çekirdek** her zaman bellektedir, işlemcinin ayrıcalıklı bir modunda çalışır (bkz. Bölüm 3.4) ve donanımla doğrudan konuşur.
- **Sistem programları** işletim sistemiyle birlikte gelir, ama sıradan programlar olarak çalışır. Sistemi kullanılabilir kılarlar.

- **Uygulama programları**, asıl işinizi yapmak için kurduğunuz her şeydir.

Tam anlamıyla "Linux" yalnızca bir çekirdektir. Ubuntu ya da Fedora gibi bir *Linux dağıtımı*, buna çoğu GNU Projesi'nden gelen binlerce sistem programı ekler.

> **Web tarayıcısı işletim sisteminin parçası mı?**
> 1998'de Amerika Birleşik Devletleri hükümeti Microsoft'a dava açtı; davanın bir kısmı şirketin Internet Explorer'ın Windows'un ayrılmaz bir parçası olduğu iddiasıyla ilgiliydi. Mahkemeler Microsoft'un tekelini yasa dışı yollarla sürdürdüğüne hükmetti ve "işletim sistemine ne dahildir?" sorusu teknik olduğu kadar hukuki bir soru hâline geldi. Kullanışlı bir pratik kural: çekirdek kesinlikle işletim sistemidir; çekirdeğin ötesinde "işletim sistemi", üreticinin standart sistem olarak sunduğu her şeydir.

2 İşletim Sisteminin Tasarım Hedefleri

William Stallings bir işletim sisteminin amaçlarını üç kelimeyle özetler.

1. **Kullanım kolaylığı (convenience).** İşletim sistemi bilgisayarı kullanmayı ve programlamayı kolaylaştırır: dosyalar, pencereler, bir kabuk, basit sistem çağrıları. Kullanıcıların donanımı bilmesi gerekmez.
2. **Verimlilik (efficiency).** İşletim sistemi donanımı iyi kullanır: işlemci yararlı işle meşgul, bellek yararlı veriyle dolu, aygıtlar paralel çalışır. İşletim sisteminin kendi **ek yükü** (overhead) düşük olmalı, yani kendisi az kaynak tüketmelidir.
3. **Gelişebilirlik (ability to evolve).** Yeni donanım (SSD'ler, GPU'lar, 5G modemler) ve yeni hizmetler, mevcut programları bozmadan eklenebilmelidir. İşletim sistemi *yapısının* seçimini her şeyden çok bu hedef belirler (bkz. Bölüm 5).

2.1 Hedefler çatışır

Üç hedef farklı yönlere çeker ve farklı bilgisayar türleri bu hedeflere farklı ağırlık verir.

Sistem türü	Ana öncelik	Neden
Kişisel bilgisayar	Kullanım kolaylığı	Her şeyin "öylece çalışmasını" isteyen tek bir kullanıcı; işlemci zaten çoğu zaman boştadır.
Sunucu / bulut	Verimlilik	Binlerce kullanıcı; %1 daha fazla iş hacmi, yüzlerce makine daha az demek olabilir.
Akıllı telefon	Verimlilik (enerji)	Pil ömrü: işletim sistemi uygulamaları ve işlemciyi agresif biçimde uyutur.
Gömülü / IoT	Verimlilik (boyut, maliyet)	Bir çamaşır makinesi denetleyicisinin birkaç kilobaytlık belleği olabilir; hiç işletim sistemi olmayabilir.
Gerçek zamanlı (frenler, dronlar)	Öngörülebilirlik	Geç gelen yanıt yanlış yanıttır: son tarihler (deadline) her zaman karşılanmalıdır.

Kolaylık ve verimlilik

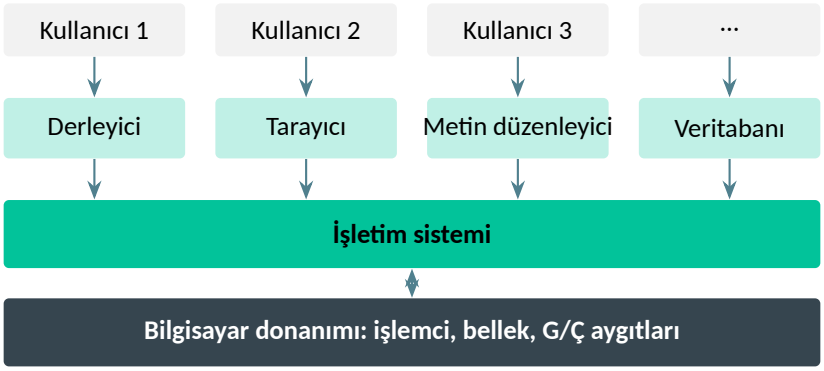
Bir program bir dosyadan 10^6 sayı okumalıdır. Her sistem çağrısının (çekirdeğe yapılan bir istek; bkz. Bölüm 4) saf ek yükünün $t_s \approx 0.1\ \mu$s olduğunu varsayalım.

- *Kolay yol:* işletim sisteminden her seferinde bir sayı istemek: 10^6 çağrı, ek yük $10^6 \times 0.1\ \mu\text{s} = 0.1\ \text{s} = 100$ ms.
- *Verimli yol:* her biri yaklaşık 10^3 sayı içeren 4096 baytlık bloklar istemek: 10^3 çağrı, ek yük $10^3 \times 0.1\ \mu\text{s} = 0.1$ ms.

Verimli yolun ek yükü 1000 kat daha azdır. C standart kütüphanesi ikisini birden sunar: `fgetc()` fonksiyonunu karakter karakter çağırırsınız, ama kütüphane arka planda büyük bloklar okur. Bu tekniğe **tamponlama** (buffering) denir.

3 Bilgisayar Sisteminin Bileşenleri

Bir bilgisayar sistemi dört bileşene ayrılabilir.

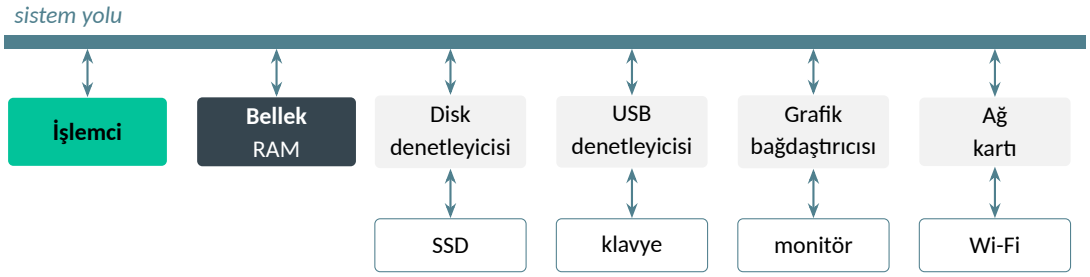


Şekil 2: Bir bilgisayar sisteminin dört bileşeni.

- **Donanım** temel hesaplama kaynaklarını sağlar: işlemci, bellek, girdi/çıktı (G/Ç; input/output, I/O) aygıtları.
- **Uygulama programları** bu kaynakları kullanıcıların sorunlarını çözmek için kullanır.
- **Kullanıcılar** insanlardır, ama aynı zamanda başka makineler ve programlar da olabilir (ör. bir web sunucusunun "kullanıcıları" başka bilgisayarlardır).
- **İşletim sistemi** donanımı denetler ve onun uygulamalar ile kullanıcılar arasında kullanımını eşgüdümler.

3.1 Donanımın düzeni

Modern bir bilgisayarda bir ya da daha fazla işlemci ve birkaç **aygıt denetleyicisi** (device controller) bulunur; hepsi ortak bir **yol** (bus) üzerinden paylaşılan ana belleğe bağlıdır.



Şekil 3: İşlemci, bellek ve aygıt denetleyicileri sistem yolunu paylaşır.

Her aygıt denetleyicisinin küçük bir yerel tamponu vardır ve işlemciyle *paralel* çalışır. İşletim sistemi her denetleyici için bir **aygıt sürücüsü** (device driver) içerir: denetleyicinin komutlarını anlayan ve işletim sisteminin geri kalanına tek tip bir arayüz sunan bir yazılım parçası.

3.2 Kesmeler

İşlemci bir aygıtın işini bitirdiğini nasıl anlar? Sürekli sorabilir ("yeni bir şey var mı?"); bu tekniğe **yoklama** (polling) denir, ama işlemci zamanını boşa harcar. Bunun yerine aygıt, bir **kesme** (interrupt) üreterek işlemciye *haber verir*:

1. Aygıt denetleyicisi bir işlemi bitirir (ör. diskten gelen veri artık kendi tamponundadır).
2. İşlemciye bir kesme hattı üzerinden sinyal gönderir.
3. İşlemci çalıştırmakta olduğu programın durumunu kaydeder ve adresi **kesme vektöründe** (interrupt vector) saklanan **kesme işleyicisine** (interrupt handler) atlar. Kesme vektörü, aygıt numarasına göre sıralanmış bir tablodur.
4. İşleyici (işletim sisteminin içindeki aygıt sürücüsünün bir parçası) olayı işler.
5. Kaydedilen durum geri yüklenir ve kesilen program hiçbir şey olmamış gibi devam eder.

Modern işletim sistemleri **kesme güdümlüdür** (interrupt-driven): hiçbir program çalışmak istemiyorsa ve hiçbir aygıt ilgi beklemiyorsa işletim sistemi yalnızca bekler. Her tuş vuruşu, fare hareketi, ağ paketi ve biten her disk okuması bir kesme olarak gelir; çoğu zaman saniyede binlerce kez.

Neden yalnızca yoklama yapmıyoruz?

Bir klavye saniyede en fazla yaklaşık 10 tuş vuruşu üretir. Klavyeyi bir kez kontrol etmenin $t_{\text{check}} = 1\ \mu\text{s}$ sürdüğünü ve tepkili hissettirmek için saniyede $f = 1000$ kez yokladığımızı varsayalım. Yoklamaya harcanan işlemci zamanı oranı

$$f \cdot t_{\text{check}} = 1000\ \text{s}^{-1} \times 10^{-6}\ \text{s} = 10^{-3} = 0.1\%$$

olur. Bu küçük görünür; ama kontrollerin $\%99$'u hiçbir şey bulmaz ve bir bilgisayarda düzinelerce aygıt vardır. Kesmelerle işlemci klavyeye yalnızca gerçekten bir tuşa basıldığında, yani saniyede yaklaşık 10 kez zaman ayırır.

3.3 Bellek hiyerarşisi

Depolama birimleri hız, boyut, maliyet ve *uçuculuk* (volatility; içeriğin güç kesildiğinde korunup korunmadığı) bakımından farklılaşan birçok türde gelir.

Düzey	Tipik boyut	Erişim süresi	Yazmaç erişimi 1 s sürseydi…
İşlemci yazmaçları	< 1 KB	≈ 0.3 ns	1 saniye
Önbellek (L1–L3)	birkaç MB	1–40 ns	3 saniye ile 2 dakika arası
Ana bellek (RAM)	8–64 GB	≈ 100 ns	≈ 5 dakika
Katı hâl sürücüsü (SSD)	0.5–4 TB	$\approx 50\ \mu$s	≈ 2 gün
Sabit disk sürücüsü (HDD)	1–20 TB	≈ 5 ms	≈ 6 ay

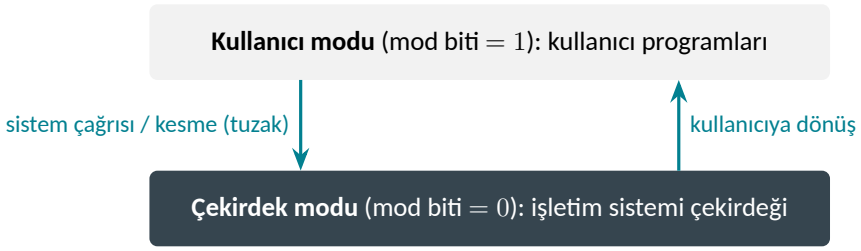
Son sütun, her erişim süresini aynı $1/(0.3\text{ ns}) \approx 3.3 \times 10^9$ çarpanıyla çarpar; örneğin bir sabit disk için $5\text{ ms} \times 3.3 \times 10^9 \approx 1.7 \times 10^7$ s ≈ 190 gün. Yazmaçlar, önbellek ve RAM **uçucudur** (volatile); SSD'ler ve diskler **kalıcıdır** (non-volatile). İşletim sisteminin işinin büyük kısmı, bir programın ihtiyaç duyduğu veri gerektiği anda hızlı bellekte olsun diye veriyi düzeyler arasında taşımaktır. Bu konuya 6. ve 7. haftalarda döneceğiz.

3.4 İkili mod çalışması

Her program her şeyi yapabilseydi, hatalı ya da kötü niyetli tek bir program diğer bütün programları yok edebilirdi. Bu yüzden koruma *donanım* desteği gerektirir. İşlemcinin bir **mod biti** (mode bit) vardır:

- **Çekirdek modu** (kernel mode; gözetmen, sistem ya da ayrıcalıklı mod da denir; mod biti $= 0$): her komut serbesttir. Bu modda yalnızca işletim sistemi çekirdeği çalışır.
- **Kullanıcı modu** (user mode; mod biti $= 1$): doğrudan G/Ç, bellek korumasını değiştirmek ya da işlemciyi durdurmak gibi **ayrıcalıklı komutlar** (privileged instructions) yasaktır.

Bir kullanıcı programı ayrıcalıklı bir komut denerse donanım bu komutu yürütmez; bunun yerine işletim sistemine bir *tuzak* (trap) üretir ve işletim sistemi genellikle programı sonlandırır. Bir programın ayrıcalıklı bir hizmete ihtiyacı olduğunda bir **sistem çağrısı** (system call) yapar: bu, çekirdek moduna geçen ve çekirdekteki sabit bir giriş noktasına atlayan özel bir komuttur. Çekirdek işini bitirince kullanıcı moduna geri geçer ve programa döner.



Şekil 4: Kullanıcı modu ile çekirdek modu arasında geçiş.

x86 işlemcilerde bu modlara *halka* (ring) denir ve 0 (en ayrıcalıklı) ile 3 arasında numaralandırılır; Linux ve Windows yalnızca halka 0'ı (çekirdek) ve halka 3'ü (kullanıcı) kullanır. Bölüm 5.1'deki MS-DOS örneği, bu koruma olmadığında ne olduğunu gösterir.

4 Uygulama: Bir Programın Çekirdeği Çağırmasını İzlemek

Bu bölümdeki programlar, bu notların yanındaki `code/` klasöründedir. Linux için C dilinde yazılmışlardır. Onları laboratuvar bilgisayarlarında, WSL'de (Windows Subsystem for Linux) ya da çevrim içi bir C derleyicisinde derleyebilirsiniz. Bu derste C yazmanız *beklenmiyor*; açıklamaları okuyun ve programın ne yaptığına odaklanın.

4.1 Örnek 1: kütüphane çağrısı ve sistem çağrısı

```c
/* hello_syscall.c -- aynı mesajı iki farklı yolla yazdırmak.
 * Derleme   : gcc hello_syscall.c -o hello_syscall
 * Çalıştırma: ./hello_syscall
 * İzleme    : strace ./hello_syscall   (yalnızca Linux)
 */
#include <stdio.h>   /* printf: kütüphane fonksiyonu */
#include <unistd.h>  /* write: sistem çağrısı sarmalayıcısı */

int main(void)
{
    /* 1) Kütüphane çağrısı: printf metni kullanıcı modunda
     *    biçimlendirir, sonra write() ile çekirdeğe yazdırır. */
    printf("printf ile merhaba!\n");
    fflush(stdout);

    /* 2) Doğrudan sistem çağrısı: 1 numaralı dosya tanımlayıcısı
     *    = ekran (standart çıktı); 21 = yazılacak bayt sayısı. */
    write(1, "write() ile merhaba!\n", 21);

    return 0;
}
```

`printf()` bir *kütüphane fonksiyonudur*: tamamen kullanıcı modunda çalışır ve metni biçimlendirir. Karakterleri gerçekten ekrana koymak için çekirdekten `write()` sistem çağrısıyla istekte bulunmak zorundadır. `strace` programı, başka bir programın yaptığı her sistem çağrısını gösterir:

```
$ gcc hello_syscall.c -o hello_syscall
$ strace ./hello_syscall
...
write(1, "Hello from printf!\n", 19)  = 19
write(1, "Hello from write()!\n", 20) = 20
exit_group(0)                         = ?
```

İki satır da sonunda aynı `write` sistem çağrısına dönüşür. Sayıların anlamı şöyledir: dosya tanımlayıcısı (file descriptor) 1 (standart çıktı, yani ekran), yazılacak baytlar, bunların sayısı ve = işaretinden sonra çekirdeğin döndürdüğü değer (gerçekten yazılan bayt sayısı).

Basit bir komut bile çok sayıda sistem çağrısı yapar. Test makinemizde `strace -c ls /` komutu 76 sistem çağrısı bildirdi; bunların çoğu dosyaları açmak, okumak, belleğe eşlemek ve kapatmak içindi.

4.2 Örnek 2: bir sistem çağrısı ne kadar pahalı?

```c
/* syscall_cost.c -- çekirdeğe bir yolculuk ne kadar pahalı?
 * Küçük, sıradan bir fonksiyonu N kez çağırırız; ardından küçük bir
 * sistem çağrısını (getpid) N kez yaparız ve süreleri karşılaştırırız.
 * Derleme   : gcc -O2 syscall_cost.c -o syscall_cost
 * Çalıştırma: ./syscall_cost
 */
#define _GNU_SOURCE
#include <stdio.h>
#include <time.h>
#include <unistd.h>
#include <sys/syscall.h>

#define N 1000000L

/* noinline: derleyicinin çağrıyı optimize edip yok etmesini engeller */
__attribute__((noinline)) long plain_function(long x)
{
    return x + 1;
}

static double seconds(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1e9;
}

int main(void)
{
    volatile long sink = 0;            /* sonuçları "kullanılmış" tutar */
    double t0, t1, t2;

    t0 = seconds();
    for (long i = 0; i < N; i++)
        sink + = plain_function(i);     /* kullanıcı modunda kalır */
    t1 = seconds();
    for (long i = 0; i < N; i++)
        sink + = syscall(SYS_getpid);   /* kullanıcı -> çekirdek -> kullanıcı */
    t2 = seconds();

    double f = (t1 - t0) / N * 1e9;    /* çağrı başına nanosaniye */
    double s = (t2 - t1) / N * 1e9;
    printf("sıradan fonksiyon çağrısı : %6.1f ns\n", f);
    printf("sistem çağrısı (getpid)   : %6.1f ns\n", s);
    printf("sistem çağrısı yaklaşık %.0f kat daha yavaş\n", s / f);
    return 0;
}
```

Test makinemizdeki (bir Linux sanal makinesi) örnek çıktı:

```
$ gcc -O2 syscall_cost.c -o syscall_cost
$ ./syscall_cost
ordinary function call :    1.5 ns
system call (getpid)   :  116.1 ns
system call is about 76 times slower
```

Sizin sayılarınız farklı olacaktır: tipik bir bilgisayarda en küçük sistem çağrısı kabaca 50–200 ns, bir fonksiyon çağrısı ise yaklaşık 1–2 ns sürer. Asıl ders orandadır: kullanıcı modu ile çekirdek modu arasındaki sınırı geçmek *pahalıdır*, çünkü işlemci yazmaçları kaydetmeli, mod değiştirmeli, isteği denetlemeli ve geri dönmelidir. Bunu Bölüm 5.4 için aklınızda tutun.

Bu ne kadar ek yük eder?

$t_{\text{func}} = 1.5$ ns ve $t_{\text{sys}} = 116$ ns iken, fonksiyon çağrıları yerine $N = 10^6$ sistem çağrısı yapmak

$$N\,(t_{\text{sys}} - t_{\text{func}}) = 10^6 \times (116 - 1.5)\ \text{ns} \approx 1.1 \times 10^8\ \text{ns} \approx 0.11\ \text{s}$$

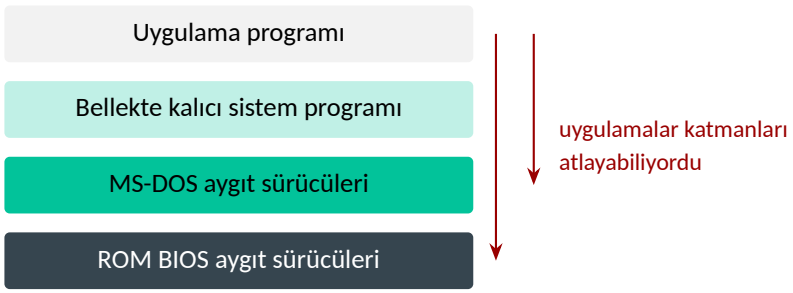
ek süre getirir.

5 İşletim Sistemi Yapıları

Genel amaçlı bir işletim sistemi çok büyük bir programdır; yalnızca Linux çekirdeği 30 milyonu aşkın kaynak kod satırından oluşur ve bunun çoğu aygıt sürücüleridir. Böyle bir programın nasıl düzenlendiği, hızını, güvenilirliğini ve gelişebilirliğini büyük ölçüde etkiler. Altı yaklaşıma bakacağız.

5.1 Basit yapı: MS-DOS

MS-DOS (1981), özgün IBM PC'de en az yerde en çok işlevi sunmak için yazıldı. Özenle ayrılmış modüllere bölünmemişti.

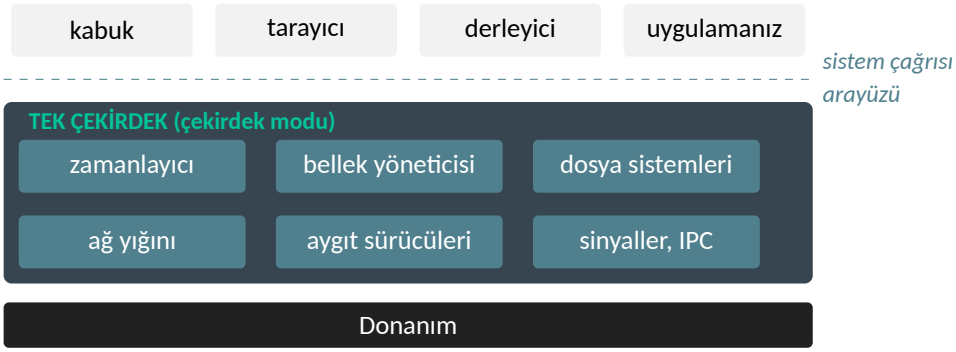


Şekil 5: MS-DOS katman yapısı: programlar donanıma doğrudan ulaşabiliyordu.

Uygulama programları işletim sistemini atlayıp doğrudan ekrana ya da diske yazabiliyordu. IBM PC'nin Intel 8088 işlemcisinde ikili mod *yoktu*; bu yüzden işletim sisteminin onları durdurmasının hiçbir yolu yoktu. Sonuç olarak tek bir hatalı program bütün sistemi çökertebiliyordu. **Ders:** koruma, hem donanım desteği hem de bu desteği kullanan bir işletim sistemi gerektirir.

5.2 Monolitik çekirdek: UNIX ve Linux

Monolitik (monolithic) bir çekirdekte tüm işletim sistemi hizmetleri (süreç zamanlama, bellek yönetimi, dosya sistemleri, ağ, aygıt sürücüleri) çekirdek modunda, tek bir adres uzayında çalışan büyük bir program oluşturur.



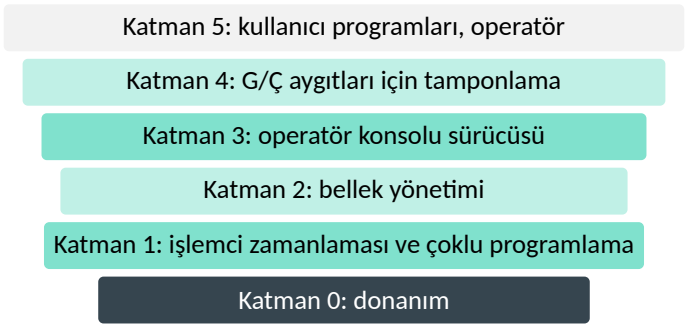
Şekil 6: Monolitik bir çekirdek.

+ **Hızlı:** çekirdek bileşenleri birbirini sıradan fonksiyon çağrıları gibi doğrudan çağırır; bir sistem çağrısı yalnızca iki mod geçişi gerektirir.
– **Bakımı zor:** tek bir programda milyonlarca satır.
– **Kırılgan:** herhangi bir parçadaki (çoğunlukla bir sürücüdeki) hata bütün çekirdeği çökertebilir (Linux'ta *çekirdek paniği* (kernel panic), Windows'ta "mavi ekran").

Örnekler: özgün UNIX (1970'ler), Linux, geleneksel BSD.

5.3 Katmanlı yaklaşım

İşletim sistemi katmanlara bölünür. 0. katman donanım, en üst katman ise kullanıcı arayüzüdür. Her katman yalnızca altındaki katmanların hizmetlerini kullanabilir.



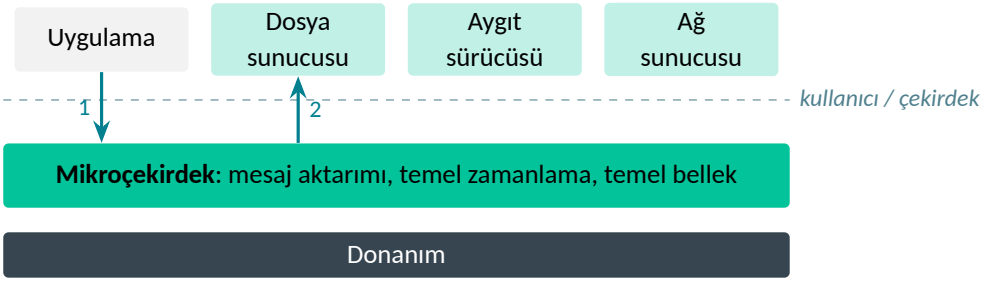
Şekil 7: Dijkstra'nın THE sistemine (1968) gevşek biçimde dayanan katmanlı bir işletim sistemi.

+ İnşa etmesi ve hata ayıklaması kolaydır: önce 1. katmanı test edin, sonra onun üzerinde 2. katmanı, ve böyle devam edin.
– Katmanların sırasına karar vermek zordur. Bellek yönetimi zamanlayıcıya ihtiyaç duyar (bir diski beklemek için), ama zamanlayıcı da belleğe ihtiyaç duyar (tabloları için). Hangisi önce gelmeli?
– Her istek, her biri ek yük getiren birçok katmandan geçer.

Saf katmanlı yapı bugün genel amaçlı sistemlerde nadirdir, ama fikir her işletim sisteminin içinde yaşar; örneğin ağ yığını kesin biçimde katmanlıdır.

5.4 Mikroçekirdek

Mikroçekirdek (microkernel) yaklaşımı, temel olmayan her şeyi çekirdekten çıkarır ve *sunucu* (server) adı verilen sıradan kullanıcı modu programları olarak çalıştırır. Mikroçekirdeğin kendisi yalnızca asgari süreç ve bellek yönetimini ve her şeyden önce programlar arasında **mesaj aktarımını** (message passing) sağlar.



Şekil 8: Bir mikroçekirdekte, uygulamanın dosya sunucusuna isteği çekirdek üzerinden bir mesaj olarak gider.

+ **Güvenilir:** çöken bir sürücü yalnızca çöken bir kullanıcı programıdır; sistem onu yeniden başlatabilir.
+ **Güvenli ve genişletilebilir:** yeni hizmetler yeni kullanıcı programlarıdır; küçük çekirdeği denetlemek daha kolaydır. seL4 mikroçekirdeğinin doğruluğu *matematiksel olarak kanıtlanmıştır*.
− **Daha yavaş:** her hizmet isteği ek mesajlar ve mod geçişleri gerektirir.

Örnekler: Mach (Carnegie Mellon, 1980'ler), MINIX 3, QNX (otomobillerde ve tıbbi cihazlarda kullanılır), L4 ve seL4.

Mikroçekirdeğin bedeli

Tek bir dosya okuması için mod geçişlerini sayalım.

- Monolitik: uygulama → çekirdek → uygulama: 2 geçiş.
- Mikroçekirdek: uygulama → çekirdek → dosya sunucusu → çekirdek → uygulama: 4 geçiş ve ayrıca 2 mesaj kopyası.

Temsilî değerler alalım: bir mod geçişi $t_s = 0.1\ \mu$s, bir mesaj kopyası $t_c = 0.15\ \mu$s, yararlı iş $t_w = 1\ \mu$s. Bu durumda

$$T_{\text{mono}} = 2t_s + t_w = 0.2 + 1.0 = 1.2\ \mu\text{s}, \qquad T_{\mu} = 4t_s + 2t_c + t_w = 0.4 + 0.3 + 1.0 = 1.7\ \mu\text{s}$$

olur; yani mikroçekirdek bu işlem için $T_{\mu}/T_{\text{mono}} \approx 1.42$ kat, başka bir deyişle yaklaşık %42 daha yavaştır. Uzun işlemlerde (büyük t_w) oran 1'e yaklaşır ve ek yük pek önemli olmaz:

$$\lim_{t_w \to \infty} \frac{4t_s + 2t_c + t_w}{2t_s + t_w} = 1.$$

5.5 Modüler çekirdek: yüklenebilir çekirdek modülleri

Modern sistemlerin çoğu **modüler** (modular) bir tasarım kullanır: bir temel çekirdek ve sistem çalışırken eklenip çıkarılabilen **yüklenebilir çekirdek modülleri** (loadable kernel modules). Bir USB aygıtı taktığınızda

Linux doğru sürücü modülünü kendiliğinden yükleyebilir.

+ Monolitik çekirdekte olduğu gibi modüller çekirdek modunda çalışır; bu yüzden aralarındaki çağrılar hızlıdır.
+ Mikroçekirdekte olduğu gibi sistem esnektir: yalnızca gereken modüller yüklenir.
− Hatalı bir modül yine çekirdek modunda çalışır ve sistemi yine çökertebilir.

Örnekler: Linux, Solaris, modern BSD'ler.

Örnek 3: en küçük çekirdek modülü. Aşağıdaki liste, standart "merhaba dünya" Linux çekirdek modülüdür. `main()` fonksiyonu yoktur; bunun yerine çekirdek, modül yüklendiğinde `hello_init()` fonksiyonunu, kaldırıldığında `hello_exit()` fonksiyonunu çağırır. Mesajları ekrana değil, çekirdek günlüğüne (kernel log) gider.

```c
/* hello_module.c -- en küçük Linux yüklenebilir çekirdek modülü (LKM).
 * Derleme : make        (çekirdek başlık dosyaları paketi gerekir)
 * Yükleme : sudo insmod hello_module.ko
 * Kontrol : lsmod | grep hello   ve   sudo dmesg | tail -2
 * Kaldırma: sudo rmmod hello_module
 * Bunu yalnızca bozmayı göze alabileceğiniz bir sanal makinede deneyin!
 */
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("MBLP131");
MODULE_DESCRIPTION("Merhaba dünya çekirdek modülü");

static int __init hello_init(void)    /* insmod sırasında çalışır */
{
    pr_info("hello_module: loaded into the kernel\n");
    return 0;                          /* 0 = başarılı */
}

static void __exit hello_exit(void)   /* rmmod sırasında çalışır */
{
    pr_info("hello_module: removed from the kernel\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

```
$ make                        # çekirdek başlık paketi gerekir
$ sudo insmod hello_module.ko   # yükle
$ lsmod | grep hello            # orada mı?
$ sudo dmesg | tail -1          # çekirdek günlüğünü oku
hello_module: loaded into the kernel
$ sudo rmmod hello_module       # kaldır
```

Önce güvenlik

Bir çekirdek modülündeki kod çekirdek modunda çalışır: bir hata bütün bilgisayarı dondurabilir. Örnek 3'ü yalnızca bir sanal makinede deneyin. Örnek burada "yüklenebilir modül"ün ne demek olduğunu *görebilmeniz* için verilmiştir; böyle bir modül yazmanız gerekmeyecek.

5.6 Hibrit sistemler

Gerçek işletim sistemleri nadiren tek bir saf tasarımı izler.

- **Windows (NT ailesi)** mikroçekirdek fikirleriyle (ayrı alt sistemler) tasarlandı, ancak hız için grafik dahil hizmetlerin çoğu çekirdek modunda çalışır. Sürücüler, modüller gibi yüklenebilir.
- **macOS ve iOS**, Mach mikroçekirdeğini BSD UNIX'in parçalarıyla birleştirip tek bir çekirdek olarak derleyen *XNU* çekirdeğini ve buna ek olarak yüklenebilir çekirdek uzantılarını kullanır.
- **Android**, değiştirilmiş bir Linux çekirdeği (monolitik ve modüler) kullanır; bunun üzerinde bir donanım soyutlama katmanı, yerel kütüphaneler ve uygulamalar için Android çalışma zamanı bulunur.

Tanenbaum–Torvalds tartışması (1992)

Ocak 1992'de MINIX mikroçekirdeğinin yazarı Andrew Tanenbaum, "LINUX is obsolete" (Linux eskimiştir) başlıklı bir mesaj yayımladı ve monolitik bir çekirdeğin "1970'lere doğru dev bir geri adım" olduğunu savundu. Linus Torvalds, Linux'un çalıştığını ve hızlı olduğunu söyleyerek yanıt verdi. Otuz yılı aşkın bir süre sonra iki tasarım da her gün kullanılıyor: Linux sunucularda ve telefonlarda, mikroçekirdekler ise otomobillerde, uçaklarda ve güvenliğin kritik olduğu diğer cihazlarda.

5.7 Karşılaştırma

Yapı	Çekirdek modunda	Hız	Güvenilirlik	Örnek
Basit	her şey (koruma yok)	yüksek	çok düşük	MS-DOS
Monolitik	tüm sistem hizmetleri	yüksek	orta	UNIX, Linux
Katmanlı	tüm katmanlar	düşük	yüksek	THE (1968)
Mikroçekirdek	yalnızca küçük çekirdek	daha düşük	yüksek	MINIX 3, QNX, seL4
Modüler	temel + yüklenen modüller	yüksek	orta	Linux, Solaris
Hibrit	değişir	yüksek	orta–yüksek	Windows, macOS, Android

Tekrarlanan ödünleşim: eklediğiniz her sınır (bir katman, bir mesaj, bir mod geçişi) güvenilirlik kazandırır ve hızdan götürür.

6 Özet

Altı cümlede 2. hafta

1. İşletim sistemi donanımı yönetir ve programlara daha basit, daha güvenli bir makine sunar; bellekte sürekli bulunan özü çekirdektir.
2. İşletim sistemine yukarıdan aşağıya bir genişletilmiş makine (soyutlama) ya da aşağıdan yukarıya bir kaynak yöneticisi (zaman ve alan çoğullama) olarak bakılabilir.
3. Üç tasarım hedefi kullanım kolaylığı, verimlilik ve gelişebilirliktir; bunlar kişisel bilgisayarlarda, sunucularda, telefonlarda ve gömülü sistemlerde farklı ağırlık taşır.
4. Bir bilgisayar sisteminin dört bileşeni vardır: donanım, işletim sistemi, uygulama programları ve kullanıcılar; aygıtlar işlemciye kesmelerle haber verir.
5. İkili mod çalışması sistemi korur; bir sistem çağrısı çekirdek moduna geçer ve bir fonksiyon çağrısından çok daha pahalıdır.
6. İşletim sistemi yapıları (basit, monolitik, katmanlı, mikroçekirdek, modüler, hibrit), hız ile güvenilirlik arasındaki farklı uzlaşmalardır.

7 Sözlük

Aygıt sürücüsü (device driver)
Bir tür aygıt denetleyicisini denetleyen işletim sistemi kodu.

Ayrıcalıklı komut (privileged instruction)
İşlemcinin yalnızca çekirdek modunda yürüttüğü komut.

Çekirdek (kernel)
İşletim sisteminin her zaman bellekte bulunan ve çekirdek modunda çalışan kısmı.

Çoğullama (multiplexing)
Bir kaynağı zamanda (sırayla) ya da alanda (bölerek) paylaştırmak.

İkili mod çalışması (dual-mode operation)
Bir mod bitiyle seçilen iki işlemci modu (kullanıcı ve çekirdek) için donanım desteği.

Kesme (interrupt)
İşlemcinin durup bir işletim sistemi işleyicisini çalıştırmasına ve sonra kaldığı yerden devam etmesine yol açan donanım sinyali.

Kesme vektörü (interrupt vector)
Kesme numarasına göre sıralanmış işleyici adresleri tablosu.

Mikroçekirdek (microkernel)
Asgari bir çekirdek; işletim sistemi hizmetlerinin çoğu, mesajlarla haberleşen kullanıcı modu sunucu programları olarak çalışır.

Monolitik çekirdek (monolithic kernel)
Tüm işletim sistemi hizmetlerinin çekirdek modunda tek bir program olarak çalıştığı çekirdek.

Sistem çağrısı (system call)
Bir programın çekirdekten yaptığı istek; işlemciyi çekirdek moduna geçirir.

Soyutlama (abstraction)
Ayrıntıları gizleyen sadeleştirilmiş bir görünüm; ör. disk sektörleri yerine bir *dosya*.

Tamponlama (buffering)
Pahalı işlemlerin sayısını azaltmak için veriyi büyük bloklarda toplamak.

Tuzak (trap)
Bir sistem çağrısının ya da bir hatanın yol açtığı, yazılım kaynaklı kesme.

Yüklenebilir çekirdek modülü (loadable kernel module)
Çalışan bir çekirdeğe eklenebilen ya da ondan çıkarılabilen kod.

8 Kendiniz Deneyin

Kendi işletim sisteminizin içine bakın

Linux (laboratuvar bilgisayarları, WSL ya da bir sanal makine):

```
$ uname -r          # çekirdek sürümü nedir?
$ lsmod | head      # hangi modüller yüklü?
$ strace -c ls      # ls'in yaptığı sistem çağrılarını say
```

Windows: Komut İstemi'nde `winver` (Windows derlemesi) ve `driverquery` (kurulu sürücüler) komutlarını çalıştırın. Görev Yöneticisi → *Performans* → CPU ekranında grafiğe sağ tıklayıp *Çekirdek sürelerini göster* seçeneğini seçin.

1. Makinenizde kaç çekirdek modülü yüklü?
2. `ls` en sık hangi sistem çağrısını kullanıyor?
3. Çekirdek modunda çok zaman geçiren bir program bulun. Bunun nedeni ne olabilir?

9 Tekrar Soruları

Kavramsal

1. İşletim sisteminin üç ders kitabı tanımını verin. Her biri hangi yönü vurgular?
2. "Genişletilmiş makine" ve "kaynak yöneticisi" bakışları arasındaki farkı açıklayın. Her birine bir örnek verin.
3. Bir zaman çoğullama ve bir alan çoğullama örneği verin.
4. Çekirdek ile bir sistem programı arasındaki fark nedir? Kabuk (shell) çekirdeğin parçası mıdır?
5. İşletim sisteminin üç tasarım hedefini sayın. (a) Bir akıllı telefon, (b) bir web sunucusu, (c) bir otomobilin fren denetleyicisi için hangisi en önemlidir?
6. Bir disk denetleyicisi kesme ürettiğinde ne olduğunu adım adım anlatın.

7. Koruma neden donanım tarafından desteklenmelidir? İkili modu olmayan bir işlemcide ne olurdu?
8. `printf()` ile `write()` arasındaki fark nedir?
9. Monolitik ve mikroçekirdek tasarımlarını karşılaştırın: her birinin bir avantajı ve bir dezavantajı.
10. Modüler bir çekirdek neden "iki dünyanın en iyisi" olarak tanımlanır? Geriye kalan zayıflığı nedir?

Sayısal

11. Bir sistem çağrısı $0.2\ \mu$s sürüyor. Bir program 1 MB'lık bir dosyayı (a) bayt bayt, (b) 4096 baytlık bloklarla okuyor. Her iki durumda sistem çağrısı ek yükünü hesaplayın. (1 MB $= 2^{20}$ bayt alın.)
12. Bir aygıt saniyede 500 kez yoklanıyor; her kontrol $2\ \mu$s sürüyor. İşlemcinin ne kadarı yoklamaya harcanır? Bu, günde kaç işlemci-saniyesi eder?
13. "Yazmaç erişimi $= 1$ s" ölçeğiyle, RAM'e (80 ns) ve bir SSD'ye ($100\ \mu$s) erişim ne kadar sürerdi? Yazmaç erişim süresini 0.3 ns alın.
14. Mikroçekirdek örneğinde yararlı iş $t_w = 10\ \mu$s'ye çıkıyor. T_μ / T_{mono} yavaşlamasını hesaplayın. Bu size ne söyler?

Sayısal soruların yanıtları

11. (a) 2^{20} çağrı $\times\ 0.2\ \mu$s ≈ 0.21 s. (b) $2^{20}/2^{12} = 256$ çağrı $\times\ 0.2\ \mu$s $\approx 51\ \mu$s, yani 4096 kat daha az.
12. $500 \times 2 \times 10^{-6} = 10^{-3} = 0.1\%$. Günde: $0.001 \times 86\,400$ s ≈ 86 işlemci-saniyesi.
13. Ölçek çarpanı $1/(0.3\text{ ns}) \approx 3.33 \times 10^9$. RAM: $80 \times 10^{-9} \times 3.33 \times 10^9 \approx 267$ s ≈ 4.4 dk. SSD: $10^{-4} \times 3.33 \times 10^9 \approx 3.3 \times 10^5$ s ≈ 3.9 gün.
14. $T_{\text{mono}} = 0.2 + 10 = 10.2\ \mu$s, $T_\mu = 0.4 + 0.3 + 10 = 10.7\ \mu$s; oran ≈ 1.05, yalnızca %5 daha yavaş. Mikroçekirdeğin ek yükü kısa ve sık işlemlerde önemlidir, uzun işlemlerde değil.

10 İleriye Bakış: 3. Hafta

- İşletim sisteminin kullanıcılara ve programlara sunduğu hizmetler.
- Ayrıntılı sistem çağrıları: türleri, parametre aktarımı, sistem çağrısı tablosu.
- Süreç (process) kavramı: yürütülmekte olan program ve bellek düzeni.
- Süreç durumları: yeni, hazır, çalışıyor, bekliyor, sonlandı; süreç denetim bloğu (process control block, PCB).

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley, Bölüm 1–2.
- W. Stallings, *Operating Systems: Internals and Design Principles*, 9. baskı, Pearson, Bölüm 2.
- A. S. Tanenbaum ve H. Bos, *Modern Operating Systems*, 4. baskı, Pearson, Kısım 1.1 ve 1.7.
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Bölüm 6, "Limited Direct Execution": `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-mechanisms.pdf`